SQL – Common Table Expressions (CTEs)

Michel Ferreira

Bases de Dados

Licenciatura em Engenharia Informática e Computação, FEUP+FCUP

Agenda

CTEs – Common Table Expressions

Basic Syntax

Multiple CTEs in one query

Simple example

Recursive CTEs

Computing D’Hondt mandates allocation with a recursive CTE

What is a CTE?

CTE = Common Table Expression

A temporary, named result set defined at the start of a query using WITH.

Used to:

- Improve readability of complex queries
- Break logic into steps
- Avoid repeating subqueries

Example structure:

```
WITH cte_name AS (
    SELECT ...
)
SELECT ...
FROM cte_name;
```

Basic Syntax

```
WITH cte_name AS (
    SELECT col1, col2
    FROM table
    WHERE condition
)
SELECT *
FROM cte_name
WHERE ...
```

Key points:

- Exists only during execution of the statement
- Behaves like a temporary result table
- The final query can reference one or more CTEs

Multiple CTEs

```
WITH
  a AS (SELECT ...),
  b AS (SELECT ...),
  c AS (
      SELECT ...
      FROM a JOIN b
  )
SELECT *
FROM c;
```

Notes:

- CTEs can reference earlier CTEs
- They help organize multi-step logic
- Replace deeply nested subqueries

An Example with Autárquicas 2021 Database

Question:

Using the Autárquicas 2021 database, write an SQL query (using a CTE) that lists the winning list in each municipality of the Autonomous Region of Madeira.

Consider party lists, coalition lists, and independent lists.

Your query should return, for each municipality in Madeira, the municipality name, the short name of the winning list, its type (party, coalition, or independent), and the number of votes it obtained. Only results for the Autonomous Region of Madeira should be included.

Example Solution

```sql
WITH all_lists AS (
    -- Parties
    SELECT
        pv.municipality_code,
        p.shortname AS shortname,
        pv.votes    AS votes,
        'party'     AS list_type
    FROM party_votes pv
    JOIN party p ON p.id = pv.party_id

    UNION ALL

    -- Coalitions
    SELECT
        c.municipality_code,
        c.shortname,
        c.votes,
        'coalition'
    FROM coalition c

    UNION ALL

    -- Independents
    SELECT
        i.municipality_code,
        i.shortname,
        i.votes,
        'independent'
    FROM independent i
)
```

```sql
SELECT
    m.name AS municipality,
    a.shortname AS winner,
    a.list_type,
    a.votes
FROM municipality m
JOIN district d
  ON d.id = m.district
JOIN all_lists a
  ON a.municipality_code = m.code
WHERE d.name = 'Madeira'
  AND a.votes = (
      SELECT MAX(votes)
      FROM all_lists
      WHERE municipality_code = m.code
  )
ORDER BY m.name, a.shortname;
```

municipality	winner	list_type	votes
CALHETA	PPD/PSD	party	5047
CÂMARA DE LOBOS	PPD/PSD	party	9866
FUNCHAL	PPD/PSD.CDS-PP	coalition	26835
MACHICO	PS	party	5951
PONTA DO SOL	PS	party	2978
PORTO MONIZ	PS	party	1147
PORTO SANTO	PPD/PSD.CDS-PP	coalition	1843
RIBEIRA BRAVA	RB1	independent	4698
SANTA CRUZ	JPP	party	11932
SANTANA	CDS-PP	party	2720
SÃO VICENTE	PPD/PSD.CDS-PP	coalition	2343

Example with Two CTEs

Query: in Madeira, find each municipality's winner (CTE #1), then summarize how many municipalities each list won (CTE #2).

```
WITH all_lists AS (
    -- unify parties, coalitions, and independents into one stream
    SELECT pv.municipality_code, p.shortname, pv.votes, 'party' AS list_type
    FROM party_votes pv
    JOIN party p ON p.id = pv.party_id

    UNION ALL
    SELECT c.municipality_code, c.shortname, c.votes, 'coalition'
    FROM coalition c

    UNION ALL
    SELECT i.municipality_code, i.shortname, i.votes, 'independent'
    FROM independent i
),
```

```
winners_madeira AS (
    -- pick the top-voted list(s) in each Madeira municipality
    SELECT
        m.code AS municipality_code,
        m.name AS municipality,
        a.shortname,
        a.list_type,
        a.votes
    FROM municipality m
    JOIN district d ON d.id = m.district
    JOIN all_lists a ON a.municipality_code = m.code
    WHERE d.name = 'Madeira'
      AND a.votes = (
          SELECT MAX(votes)
          FROM all_lists
          WHERE municipality_code = m.code
      )
)
```

Example with Two CTEs (cont.)

```sql
SELECT
    shortname AS winner,
    list_type,
    COUNT(*) AS municipalities_won
FROM winners_madeira
GROUP BY shortname, list_type
ORDER BY municipalities_won DESC, shortname;
```

winner	list_type	municipalities_won
PPD/PSD.CDS-PP	coalition	3
PS	party	3
PPD/PSD	party	2
CDS-PP	party	1
JPP	party	1
RB1	independent	1

What each CTE does:

- all_lists: creates one unified table of all vote sources (party/coalition/independent).
- winners_madeira: filters to Madeira and keeps only the list(s) with max votes per municipality.
- Final query: counts wins per list in Madeira.

Recursive CTEs (WITH RECURSIVE)

Purpose:

Recursive CTEs allow a query to repeatedly apply a rule to generate successive rows, typically used for hierarchical data, graph traversal, or generating numeric sequences.

```
WITH RECURSIVE cte_name AS (
    -- 1. Anchor member: produces the initial row(s)
    SELECT ...

    UNION ALL

    -- 2. Recursive member: refers to the CTE itself
    SELECT ...
    FROM cte_name
    WHERE ...
)
SELECT * FROM cte_name;
```

- The anchor member starts the recursion.
- The recursive member repeatedly builds on previous results.
- Recursion ends when the recursive member produces no new rows.

Example: generate vote thresholds (50k - 500k)

Analyse how many parties exceed increasing vote thresholds.

- Builds the sequence 50000, 100000, …, 500000.
- For each threshold t, counts how many party lists nationwide received more than t votes.

```
WITH RECURSIVE thresholds AS (
    -- Anchor: start at 50,000 votes
    SELECT 50000 AS t

    UNION ALL

    -- Recursive step: add 50,000 until reaching 500,000
    SELECT t + 50000
    FROM thresholds
    WHERE t < 500000
),
party_totals AS (
    -- Total votes per party across ALL municipalities (national total)
    SELECT
        pv.party_id,
        SUM(pv.votes) AS total_votes
    FROM party_votes pv
    GROUP BY pv.party_id
)
```

```
SELECT
    th.t AS vote_threshold,
    COUNT(*) AS parties_above_threshold
FROM thresholds th
JOIN party_totals pt
  ON pt.total_votes > th.t
GROUP BY th.t
ORDER BY th.t;
```

vote_threshold	parties_above_threshold
50000	6
100000	5
150000	3
200000	3
250000	3
300000	3
350000	2
400000	1
450000	1
500000	1

Limitations of Recursive CTEs in SQLite

- SQLite supports WITH RECURSIVE, but with important constraints that affect how recursive queries must be written.
  1. Only one recursive reference is allowed per recursive step

```
WITH RECURSIVE t AS (
  SELECT 50000 AS x
  UNION ALL
  SELECT x + 50000
  FROM t
  WHERE x IN (SELECT x FROM t)   -- second reference ✗
)
```

  2. Recursion depth is limited: SQLite stops recursion once a configured maximum step count is reached.
  3. Recursive CTEs can be slow for large data, as SQLite evaluates recursion row-by-row.

Using a Recursive Query to Evaluate Mandates Distribution using the D’Hondt Method

- In many elections, such as in Autárquicas, the mandates for the Municipal Council are allocated using the D'Hondt method, which assigns seats one by one based on descending vote quotients.
- Let’s build a recursive D’Hondt (Hondt) query for computing the mandates distribution for the Municipality of ‘PORTO’.
- Let’s start by getting all lists and votes in ‘PORTO’.
- We first unify parties, coalitions, and independents into one list of competitors for ‘PORTO’.

D’Hondt Method Recursive Query

- Step 1) Get all lists and votes in ‘PORTO’

```
WITH lists_porto AS (
    -- Parties in Porto
    SELECT
        m.code AS municipality_code,
        p.shortname AS shortname,
        pv.votes AS votes,
        'party' AS list_type
    FROM municipality m
    JOIN party_votes pv ON pv.municipality_code = m.code
    JOIN party p ON p.id = pv.party_id
    WHERE m.name = 'PORTO'

    UNION ALL

    -- Coalitions in Porto
    SELECT
        m.code,
        c.shortname,
        c.votes,
        'coalition'
    FROM municipality m
    JOIN coalition c ON c.municipality_code = m.code
    WHERE m.name = 'PORTO'

    UNION ALL

    -- Independents in Porto
    SELECT
        m.code,
        i.shortname,
        i.votes,
        'independent'
    FROM municipality m
    JOIN independent i ON i.municipality_code = m.code
    WHERE m.name = 'PORTO'
)
SELECT * FROM lists_porto
ORDER BY votes DESC, shortname;
```

municipality_code	shortname	votes	list_type
131200	RM	41213	independent
131200	PS	18192	party
131200	PPD/PSD	17481	party
131200	PCP-PEV	7610	party
131200	BE	6321	party
131200	CH	2983	party
131200	PAN	2821	party
131200	L	461	party
131200	VP	424	party
131200	PPM	211	party
131200	E	79	party

D’Hondt Method Recursive Query (cont. I)

- Step 2) Get how many mandates ’PORTO’ elects

```
WITH porto_mandates AS (
    SELECT mandates
    FROM municipality
    WHERE name = 'PORTO'
)
SELECT * FROM porto_mandates;
```

```
mandates
--------
13
```

D’Hondt Method Recursive Query (cont. II)

- Step 3 — Using a Recursive CTE to Generate the D’Hondt Divisors
- In D’Hondt, each list competes with the sequence of divisors:

$$1,\ 2,\ 3,\ \ldots,\ M$$

  where M is the number of mandates in the municipality.
- We need these numbers (k) because each quotient is:

$$\text{quotient} = \frac{\text{votes}}{k}$$

- So our next step is to generate the divisor sequence 1, 2, 3, …, mandates.
- We do this with a recursive CTE.

D’Hondt Method Recursive Query (cont. III)

- Recursive CTE to generate the divisors

```
divisors AS (
    -- Anchor: start at 1
    SELECT 1 AS k
    UNION ALL
    -- Recursive step: 2,3,...,n (n = mandates)
    SELECT k + 1
    FROM divisors
    WHERE k < (SELECT n FROM porto_mandates)
)
```

- The anchor row produces the first divisor: 1.
- The recursive part keeps adding 1 until reaching the mandate count.
- For ‘PORTO’ (13 mandates), this emits:
- 1, 2, 3, ..., 13.
- This is the only part of the query that needs recursion.

D’Hondt Method Recursive Query (cont. IV)

- Step 4 — Generate all D’Hondt quotients
  - Once we have lists_porto and divisors, we take the Cartesian combination:

```
SELECT
    lp.shortname,
    lp.list_type,
    d.k AS divisor,
    (lp.votes * 1.0) / d.k AS quotient
FROM lists_porto lp
JOIN divisors d
```

  - We generate all possible quotient candidates for seats.
- Step 5 — Select the top N quotients (N = number of mandates)

```
SELECT shortname, list_type, divisor, quotient
FROM quotients
ORDER BY quotient DESC, shortname ASC, divisor ASC
LIMIT (SELECT n FROM porto_mandates)
```

D’Hondt Method Recursive Query (cont. V)

- Step 6 — Count seats per list
  - Finally:

```
SELECT
    shortname,
    list_type,
    COUNT(*) AS mandates_elected
FROM top_quotients
GROUP BY shortname, list_type
ORDER BY mandates_elected DESC, shortname ASC;
```

  - Each occurrence of a list in the top quotients = 1 elected mandate.
  - Group and count: gives the final distribution.
- This produces the correct D’Hondt allocation for the Municipality of Porto, using a recursive CTE only where it is logically needed (to generate the divisor sequence).

D’Hondt Method Recursive Query (cont. VI)

```
WITH RECURSIVE
lists_porto AS (
    -- Parties
    SELECT p.shortname, pv.votes, 'party' AS list_type
    FROM municipality m
    JOIN party_votes pv ON pv.municipality_code = m.code
    JOIN party p ON p.id = pv.party_id
    WHERE m.name = 'PORTO'

    UNION ALL
    -- Coalitions
    SELECT c.shortname, c.votes, 'coalition'
    FROM municipality m
    JOIN coalition c ON c.municipality_code = m.code
    WHERE m.name = 'PORTO'

    UNION ALL
    -- Independents
    SELECT i.shortname, i.votes, 'independent'
    FROM municipality m
    JOIN independent i ON i.municipality_code = m.code
    WHERE m.name = 'PORTO'
),
```

```
porto_mandates AS (
    SELECT mandates AS n
    FROM municipality
    WHERE name = 'PORTO'
),
divisors AS (
    -- Anchor: start at 1
    SELECT 1 AS k
    UNION ALL
    -- Recursive step: 2,3,...,n
    SELECT k + 1
    FROM divisors
    WHERE k < (SELECT n FROM porto_mandates)
),
quotients AS (
    -- All D'Hondt quotients for Porto
    SELECT
        lp.shortname,
        lp.list_type,
        d.k AS divisor,
        (lp.votes * 1.0) / d.k AS quotient
    FROM lists_porto lp
    JOIN divisors d
),
```

```
top_quotients AS (
    -- Keep only the top N quotients (= mandates)
    SELECT shortname, list_type, divisor, quotient
    FROM quotients
    ORDER BY quotient DESC, shortname ASC, divisor ASC
    LIMIT (SELECT n FROM porto_mandates)
)
SELECT
    shortname,
    list_type,
    COUNT(*) AS mandates_elected
FROM top_quotients
GROUP BY shortname, list_type
ORDER BY mandates_elected DESC, shortname ASC;
```

shortname	list_type	mandates_elected
RM	independent	6
PS	party	3
PPD/PSD	party	2
BE	party	1
PCP-PEV	party	1

Wooclap Time!

Any doubts?